CODING FOR DATA SCIENCE AND ANALYTICS

Exercise Session 4: Functions and Recursion

Exercise 1 — Modularization

Let's revisit the temperature conversion exercise, but this time using **functions** to make the code modular and reusable.

Conversion formulas:

- Celsius to Kelvin: $K = C + 273.15$
- Celsius to Fahrenheit: $F = C \times \frac{9}{5} + 32$

Goal

Define two separate functions:

- `to_fahrenheit(celsius)`: returns the temperature in Fahrenheit.
- `to_kelvin(celsius)`: returns the temperature in Kelvin.

Ask the user for a temperature in Celsius, call both functions, and print the returned results.

Exercise 2 — Variable Scope and References

Python arguments are passed by reference. Let's verify how this behaves with immutable objects (like numbers and strings) using the `id()` function.

Goal

Create a function `modify_data(num, text)` that:

1. Adds 1 to the integer `num`.
2. Adds the string `" updated"` to `text`.

Task

Print the `id()` of both variables in the main program **before** calling the function, **inside** the function itself (after modifying them), and **after** the function call. Observe what happens to the memory addresses and the final values.

Exercise 3 — Recursion: The Factorial

A recursive function is a function that calls itself to break down a problem into smaller instances.

Goal

Implement the factorial function $n!$ using recursion.

Mathematical rules:

- **Base case:** If $n = 0$, return 1.
- **Recursive case:** For any $n > 0$, return $n \times \text{factorial}(n - 1)$.

Test your function by computing and printing the factorial of 5 (which should return 120).